Design, Optimization, and Hardware Realization of a 3×3 Box Filter Using Pipelining and Folding Techniques

Sharvil Pingale, Advait Rao, Anish Shende
VIT Chennai

***Abstract*—This paper presents a detailed design methodology for implementing a 3×3 moving average (box) filter in both software and hardware domains. The algorithm is initially modeled and verified in Python, followed by a hardware implementation using Verilog HDL. Architectural optimizations such as pipelining and folding are applied to improve performance and reduce hardware complexity. The pipelined design enhances throughput by reducing critical path delay, whereas the folded design minimizes area by reusing computational resources. The design is synthesized using Xilinx Vivado, and performance metrics including latency, throughput, and power consumption are analyzed. The results demonstrate key trade-offs in VLSI DSP system design.**

I. Introduction

Digital image processing has become a fundamental component of modern computing systems, particularly in applications such as autonomous systems, medical imaging, and intelligent surveillance. Image filtering is one of the most essential preprocessing operations, used to remove noise and improve image quality before further analysis [2]. Among various filtering techniques, the moving average or box filter is widely used due to its simplicity and computational efficiency.

Despite its simple formulation, the box filter serves as an ideal candidate for studying hardware implementation of DSP algorithms. It allows exploration of architectural optimization techniques such as pipelining and folding, which are critical in high-performance and resource-constrained VLSI systems [1]. In this work, the filter is first implemented in Python for validation and then translated into Verilog for hardware realization. Further optimizations are applied to analyze performance and hardware trade-offs.

II. Mathematical Background

The 3×3 box filter computes the average of a pixel and its eight neighboring pixels:

$$y(i,j) = \frac{1}{9} \sum_{m=-1}^{1} \sum_{n=-1}^{1} x(i+m, j+n) \tag{1}$$

This operation is equivalent to a convolution with a uniform kernel:

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{2}$$

From a signal processing perspective, the box filter acts as a low-pass filter, attenuating high-frequency components such as noise while preserving low-frequency components. However, this smoothing effect may lead to loss of edge sharpness, which is a common limitation of averaging filters [2].

III. Design Methodology

The design flow followed in this work is structured and systematic. Initially, the algorithm is implemented in Python to verify correctness and observe its effect on image data. This is followed by a hardware implementation using Verilog HDL. Functional verification is performed through simulation, after which optimization techniques such as pipelining and folding are applied. Finally, the design is synthesized using Vivado to evaluate performance metrics. Such a structured approach ensures correctness and efficiency at each stage of the design process.

IV. Software Implementation

The box filter was implemented in Python using a sliding window technique. Each pixel is replaced by the average of its surrounding neighbors. This approach closely follows the mathematical formulation and provides a clear understanding of the filtering process.

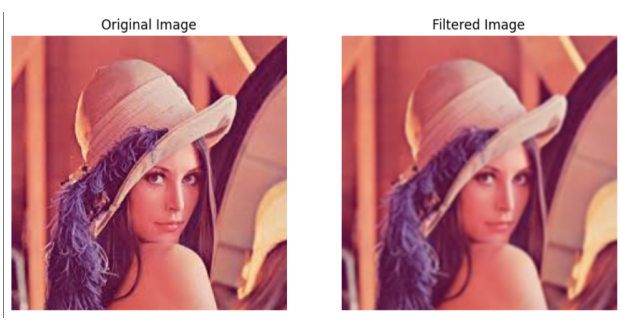

Fig. 1. Original and Filtered Image

The results confirm that the filter effectively reduces noise and smooths pixel intensity variations, validating the correctness of the algorithm.

V. Data Flow Graph Representation

The filter computation can be represented using a Data Flow Graph (DFG), where nodes represent arithmetic operations and

edges represent data dependencies. The DFG reveals that the computation requires eight additions followed by a division. This representation is essential for identifying parallelism and applying architectural transformations such as pipelining and folding [1].

VI. Hardware Implementation

The algorithm was implemented in Verilog HDL. The RTL design consists of adders, registers, and a division unit. Functional verification was performed using simulation.

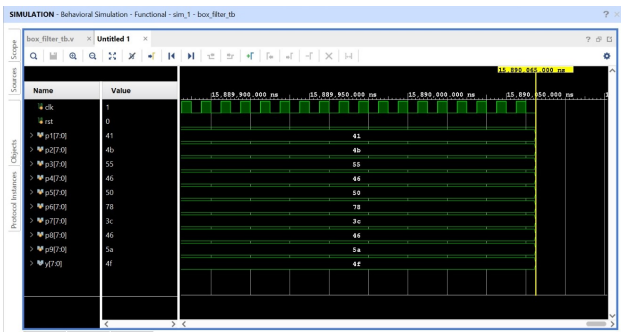

Fig. 2. Simulation Waveform

The RTL schematic illustrates the hardware structure of the design.

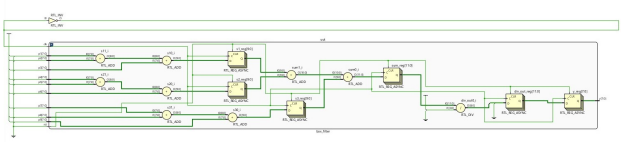

Fig. 3. RTL Schematic

VII. Pipelined Architecture

Pipelining is used to improve throughput by dividing computation into multiple stages. In this design, the computation is divided into three stages: partial sum calculation, accumulation, and normalization.

By introducing pipeline registers, the critical path delay is reduced, allowing higher clock frequencies. Once the pipeline is filled, the system produces one output per clock cycle, significantly improving performance. However, this improvement comes at the cost of increased hardware usage due to additional registers and functional units.

VIII. Folding Transformation

Folding reduces hardware complexity by reusing a single functional unit over multiple clock cycles. The folding transformation is governed by:

$$D_F(U \to V) = Nw(e) - P_U + v - u \tag{3}$$

In this design, a folding factor of 8 is used, mapping all addition operations onto a single adder. The computation is performed sequentially as:

$$S(n) = S(n-1) + p_n \tag{4}$$

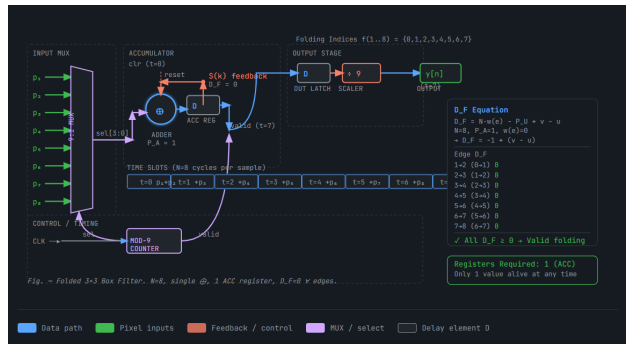

Fig. 4. Folded Architecture

While folding reduces hardware requirements, it increases latency, making it suitable for resource-constrained systems.

IX. Lifetime Analysis

Lifetime analysis is used to determine the number of registers required. The lifetime of a variable is defined as the duration between its production and consumption:

$$Lifetime = T_{output} - T_{input} \tag{5}$$

Time-Multiplexed Operation Schedule

Clock Cycle (t)	MUX Selects	Adder Operation	ACC Register	D_F
t = 0	sel = p_1, p_0	$p_1 + p_0$	$S(0) \leftarrow p_1 \cdot p_0$ *(ACC cleared first)*	0
t = 1	sel = p_2, ACC	$S(0) + p_2$	$S(1) \leftarrow S(0)+p_2$	0
t = 2	sel = p_3, ACC	$S(1) + p_3$	$S(2) \leftarrow S(1)+p_3$	0
t = 3	sel = p_4, ACC	$S(2) + p_4$	$S(3) \leftarrow S(2)+p_4$	0
t = 4	sel = p_5, ACC	$S(3) + p_5$	$S(4) \leftarrow S(3)+p_5$	0
t = 5	sel = p_6, ACC	$S(4) + p_6$	$S(5) \leftarrow S(4)+p_6$	0
t = 6	sel = p_7, ACC	$S(5) + p_7$	$S(6) \leftarrow S(5)+p_7$	0
t = 7	sel = p_8, ACC	$S(6) + p_8$	$S(7) = \Sigma p_1 \ldots p_8$ → latch valid	0
t = 8	–	$S(7) \div 9$	–	–

Fig. 5. Lifetime Analysis

The analysis shows that only one register is required, indicating efficient resource utilization.

X. Synthesis Results

The design was synthesized using Vivado, and the results show efficient utilization of hardware resources with low power consumption.

XI. Discussion

The results demonstrate that pipelining and folding are complementary techniques. Pipelining improves throughput and is suitable for high-performance applications, while folding minimizes hardware usage and is suitable for low-power systems. The choice of architecture depends on system requirements and constraints.

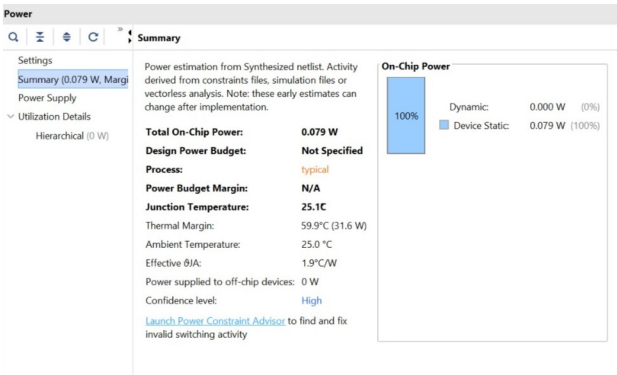

Fig. 6. Power Analysis

XII. Conclusion

This work demonstrates a complete DSP design flow from algorithm to hardware implementation. The box filter was successfully implemented and optimized using pipelining and folding techniques, highlighting important trade-offs in VLSI design.

References

[1] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Wiley, 1999.

[2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Pearson, 2018.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*. Pearson, 2010.

[4] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001.

[5] K. K. Parhi, "Algorithm transformation techniques for concurrent processor arrays," *Proceedings of the IEEE*, 1989.

[6] S. Zhang et al., "FPGA-based real-time image processing," *IEEE Trans. Consumer Electronics*, 2006.

Appendix A
Python Implementation of Box Filter

The following Python code was used to simulate the 3×3 moving average (box) filter on a colored image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = img.astype(np.float32)

k = 3
kernel = np.ones((k,k))/(k*k)

h,w,c = img.shape
output = np.zeros_like(img)

pad = k//2

for channel in range(3):
    padded = np.pad(img[:,:,channel], pad, mode='edge')
    for i in range(h):
        for j in range(w):
            window = padded[i:i+k , j:j+k]
            output[i,j,channel] = np.sum(window * k

output = np.clip(output,0,255).astype(np.uint8)

plt.subplot(121)
plt.imshow(img.astype(np.uint8))
plt.title("Original")

plt.subplot(122)
plt.imshow(output)
plt.title("Filtered")

plt.show()
```

Appendix B
Pipelined Verilog Implementation

The following Verilog module implements the 3×3 box filter using a three-stage pipelined architecture.

```
module box_filter (
    input clk,
    input rst,

    input [7:0] p1, p2, p3,
    input [7:0] p4, p5, p6,
    input [7:0] p7, p8, p9,

    output reg [7:0] y
);

reg [9:0] s1, s2, s3;
reg [11:0] sum;
reg [11:0] div_out;

// Stage 1: Row-wise addition
always @(posedge clk or posedge rst) begin
    if (rst) begin
        s1 <= 0; s2 <= 0; s3 <= 0;
    end else begin
        s1 <= p1 + p2 + p3;
        s2 <= p4 + p5 + p6;
        s3 <= p7 + p8 + p9;
    end
end

// Stage 2: Accumulation
always @(posedge clk or posedge rst) begin
    if (rst)
        sum <= 0;
    else
        sum <= s1 + s2 + s3;
end
```

```
// Stage 3: Division
always @(posedge clk or posedge rst) begin
    if (rst)
        div_out <= 0;
    else
        div_out <= sum / 9;
end

// Output stage
always @(posedge clk or posedge rst) begin
    if (rst)
        y <= 0;
    else
        y <= div_out[7:0];
end

endmodule
```

APPENDIX C
VERILOG TESTBENCH

The following testbench was used to verify the functionality of the pipelined box filter design.

```
`timescale 1ns/1ps

module box_filter_tb;

reg clk;
reg rst;

reg [7:0] p1, p2, p3;
reg [7:0] p4, p5, p6;
reg [7:0] p7, p8, p9;

wire [7:0] y;

// Instantiate DUT
box_filter uut (
    .clk(clk),
    .rst(rst),
    .p1(p1), .p2(p2), .p3(p3),
    .p4(p4), .p5(p5), .p6(p6),
    .p7(p7), .p8(p8), .p9(p9),
    .y(y)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    // Reset
    #10 rst = 0;

    // Test case 1
    p1=8'd10; p2=8'd20; p3=8'd30;
    p4=8'd40; p5=8'd50; p6=8'd60;
    p7=8'd70; p8=8'd80; p9=8'd90;
    #20;

    // Test case 2
    p1=8'd5; p2=8'd15; p3=8'd25;
    p4=8'd35; p5=8'd45; p6=8'd55;
    p7=8'd65; p8=8'd75; p9=8'd85;
    #20;

    // Test case 3
    p1=8'd100; p2=8'd110; p3=8'd120;
    p4=8'd130; p5=8'd140; p6=8'd150;
    p7=8'd160; p8=8'd170; p9=8'd180;
    #50;

    $stop;
end

endmodule
```